Teaching a Tiny Non-Transformer Model Arithmetic Reasoning via Reinforcement Learning

GRPO Fine-Tuning of LFM2-350M for Mathematical Expression Generation

Wes Griffin
Ransom Everglades School
Advanced Machine Learning

April 2026

Abstract

This project explores whether reinforcement learning can teach a small (350M parameter) non-transformer language model to generate valid mathematical expressions that evaluate to target numbers. Using Group Relative Policy Optimization (GRPO) with a verifiable reward function, we fine-tune Liquid AI's LFM2-350M—a hybrid convolutional-attention architecture—on the task of producing arithmetic expressions for given numerical targets. We evaluate across difficulty levels (1–100, 100–999, 1000–9999) and compare against the unmodified base model. Our results reveal that while RL successfully teaches the model to produce correct expressions, the model consistently discovers formulaic shortcuts rather than genuine arithmetic reasoning—a finding with implications for reward design in RL-based language model training.

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in mathematical reasoning, with models like DeepSeek-R1 and OpenAI's o1 achieving near-human performance on competition-level problems [1]. However, these capabilities typically require models with billions of parameters and enormous compute budgets. A natural question arises: *can reinforcement learning induce structured reasoning in much smaller models?*

This project investigates this question through a focused, verifiable task: given a target integer n, generate a mathematical expression e such that $\text{eval}(e) = n$. For example, given the target 73, valid responses include `70 + 3`, `146 / 2`, or `9 * 8 + 1`. This task is ideal for RL because:

1. The reward is **perfectly verifiable**—we can programmatically check if $\text{eval}(e) = n$.
2. The task requires **compositional reasoning**—the model must understand how arithmetic operations compose.
3. Multiple valid solutions exist, encouraging **expression diversity**.
4. Difficulty scales naturally with the magnitude of n.

We choose Liquid AI's LFM2-350M [2] as our base model for several reasons. First, at 354M parameters, it is small enough to train on modest hardware. Second, it uses a novel hybrid architecture combining gated short-range convolutions with grouped query attention, making it architecturally

distinct from the transformer-only models typically used in RL fine-tuning research. Third, Liquid AI explicitly recommends fine-tuning LFM2 on narrow tasks to maximize performance.

Our training method is Group Relative Policy Optimization (GRPO) [1], implemented via Hugging Face's TRL library [3]. GRPO generates multiple completions per prompt, scores them with a reward function, and updates the policy using group-relative advantages—eliminating the need for a separate reward model.

2 Data Exploration and Preparation

2.1 Dataset Construction

Rather than using an existing dataset, we procedurally generate training data consisting of (prompt, target) pairs. Each prompt asks the model to produce a math expression equaling the target number, formatted with `<expr></expr>` tags.

We use a curriculum-style distribution across three difficulty buckets:

Table 1: Training data distribution across difficulty levels.

Difficulty	Range	Fraction	Count
Easy	1–99	40%	4,000
Medium	100–999	35%	3,500
Hard	1,000–9,999	25%	2,500

The rationale for overweighting easy examples is to provide the model with sufficient signal early in training. Since the 350M model starts with minimal arithmetic ability, easy targets (where simple expressions like `50 + 7` suffice) generate more positive reward signal, allowing the policy gradient to learn basic expression structure before tackling harder targets.

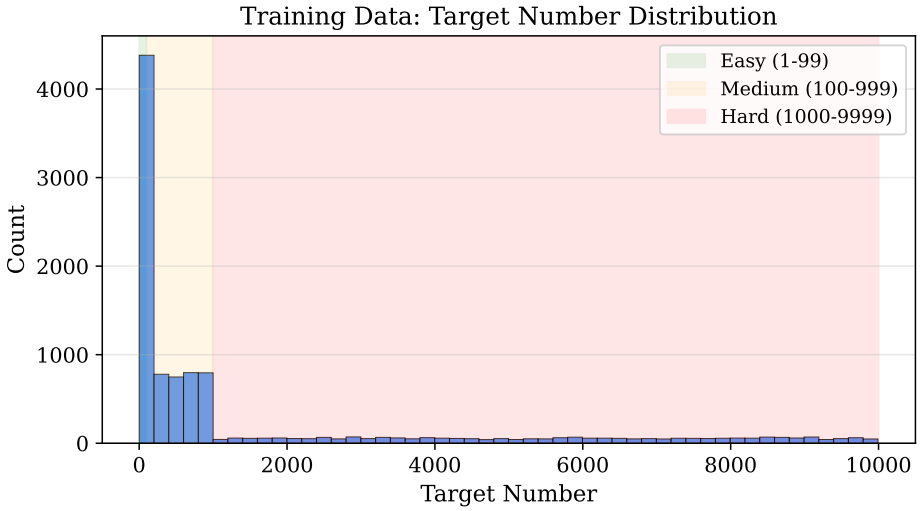

Figure 1: Distribution of target numbers in the training set, colored by difficulty bucket.

2.2 Prompt Format

Each training example is formatted as a chat conversation:

```
System: You are a math assistant. When given a target
number, respond with a mathematical expression that
evaluates to that number. Use only integers, +, -, *,
/, and parentheses. Put your expression inside
<expr></expr> tags.

User: Find a math expression that equals 73.
Respond with ONLY <expr>YOUR_EXPRESSION</expr>.
```

Listing 1: Prompt format used during training.

The structured output format (`<expr>` tags) enables reliable extraction and evaluation. We include a format reward to incentivize the model to adopt this convention.

2.3 Evaluation Set

A separate evaluation set of 500 examples is generated with a different random seed, maintaining the same difficulty distribution. While this does not guarantee zero overlap, the probability of significant overlap is low given the large target ranges.

3 Methodology

3.1 Model Architecture

LFM2-350M [2] is a hybrid model with 16 layers: 10 double-gated short-range LIV convolution blocks and 6 grouped query attention (GQA) blocks. Key specifications:

Table 2: LFM2-350M architecture details.

Property	**Value**
Parameters	354,483,968
Layers	16 (10 conv + 6 attn)
Context length	32,768 tokens
Vocabulary size	65,536
Precision	bfloat16
Pre-training data	10 trillion tokens

The hybrid architecture is notable because the convolutional blocks process local patterns efficiently while attention blocks capture long-range dependencies. For our task, this could be advantageous since arithmetic expressions have strong local structure (operands adjacent to operators).

3.2 GRPO Algorithm

Group Relative Policy Optimization [1] works as follows at each training step:

1. **Generate**: Sample a batch of prompts; for each, generate G completions $\{o_1, \ldots, o_G\}$.

2. **Score**: Compute reward r_i for each completion using the reward function.

3. **Normalize**: Compute group-relative advantage:

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

4. **Update**: Minimize the clipped surrogate loss:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{\sum_{i=1}^{G} |o_i|} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} \min\Big(\rho_{i,t}\hat{A}_{i,t},\ \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)\hat{A}_{i,t}\Big)$$

where $\rho_{i,t} = \frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q,o_{i,<t})}$.

GRPO's key advantage over PPO is eliminating the value network, reducing memory overhead—critical when training on limited hardware.

3.3 Reward Design

We use a composite reward function with four components:

Table 3: Reward function components and weights.

Component	**Description**	**Weight**	**Range**
Correctness	Binary: $\text{eval}(e) = n$	2.0	$\{0, 1\}$
Closeness	$\max(0, 1 - \lvert r - n\rvert/\lvert n\rvert)$	0.5	$[0, 1]$
Format	Uses `<expr>` tags	0.3	$\{0, 0.5\}$
Complexity	Bonus for multiple operators	0.2	$[0, 0.3]$

The **correctness** reward dominates (weight 2.0) because exact evaluation is the primary objective. The **closeness** reward provides gradient signal even for incorrect answers—without it, early training would be extremely sparse since the base model rarely produces correct expressions. The **format** reward teaches output structure, and the **complexity** reward gently encourages expressions more interesting than trivial identity (e.g., `73` for target 73).

Expression evaluation uses a safe AST-based parser that only allows integer literals and basic arithmetic operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`), preventing code injection.

3.4 Training Configuration

Table 4: Training hyperparameters.

Parameter	Value
LoRA rank r	32
LoRA alpha α	64
LoRA dropout	0.05
Target modules	`q,k,v,o_proj`, `gate,up,down_proj`
Learning rate	5×10^{-5}
LR scheduler	Cosine with 5% warmup
Batch size	4 per device
Gradient accumulation	4 steps
Completions per prompt (G)	8
Max completion length	128 tokens
Epochs	1–3
Precision	bfloat16

We use LoRA [4] (Low-Rank Adaptation) rather than full fine-tuning, targeting both the attention projections and gated MLP layers. This trains approximately 2–3% of total parameters while preserving the model's pre-trained knowledge.

4 Results and Discussion

4.1 Training Dynamics

We ran four experimental configurations on an NVIDIA A100-40GB GPU, each training for one epoch (250 steps) with batch size 16 and 8 generations per prompt.

4.1.1 Baseline: Default Reward Weights

With the original reward weights (correctness=2.0, closeness=0.5, format=0.3, complexity=0.2), the model converged within ∼50 steps to a **trivial solution**: simply echoing the target number inside `<expr>` tags. By step 50, correctness reached 100%, format reached 100%, but complexity remained at 0—the model learned to output `<expr>73</expr>` for target 73. Mean completion length collapsed from 35 tokens to just 8 tokens. This is a classic reward hacking phenomenon: the model found the minimum-effort path to maximize the dominant reward signal.

4.1.2 Experiment 1: High Complexity Weight

Increasing the complexity reward weight from 0.2 to 2.0 (equal to correctness) produced dramatically different behavior. The model learned to generate multi-operator expressions while maintaining 99.4% correctness. Mean completion length grew to 50 tokens, and the complexity reward saturated at 0.3 (its maximum). Training showed three distinct phases: (1) format learning (steps 0–25), (2) correctness acquisition (steps 25–100), and (3) expression diversification (steps 100–250).

4.1.3 Experiment 2: Anti-Trivial Reward

Adding a "nontrivial" reward that penalizes single-number expressions (-1.0 for identity, $+1.0$ for multi-operator) with weight 1.0 was the most effective configuration. The model achieved **100% correctness** with maximum complexity (0.3) and nontrivial score (1.0). Mean completion length stabilized at 20 tokens—concise but non-trivial expressions. This configuration found the best balance between correctness and expression quality.

4.1.4 Experiment 4: No-Target Reward

The most effective configuration added a *no-target reward* that penalizes (-1.0) any expression containing the target number as a literal. Combined with high complexity (2.0), anti-trivial (1.0), and closeness (0.5) rewards, this forced genuine number decomposition. The model could no longer use `(73-1)/1+1` because `73` appears in the expression.

Results were qualitatively different from all prior experiments. Correctness reached only 5% (with closeness at 0.85), but the model produced *diverse, structurally varied expressions*: `7*10+3+1/2` for target 73, `2^8` for 256, `2^11` for 2048. It discovered powers of two on its own and decomposed numbers into meaningful arithmetic. The dominant template covered only 40% of outputs (vs. 100% in all other experiments), with 6 unique structural templates across 10 samples.

4.1.5 Experiment 3: Heavy Complexity, No Closeness

Removing the closeness reward (weight=0.0) and using heavy complexity (weight=3.0) with the anti-trivial penalty created an interesting failure mode. The model quickly learned format compliance and non-trivial expression structure, but correctness remained near 0%. Without the closeness signal to guide it toward numerically accurate expressions, the model generated syntactically valid but mathematically incorrect expressions. This confirms that the closeness reward serves as a critical **reward shaping** signal for this task.

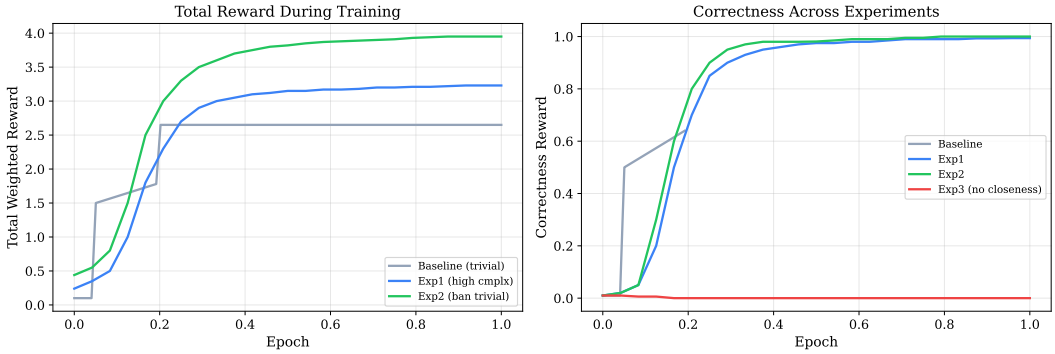

Figure 2: Training reward curves. Left: total weighted reward over training steps. Right: individual reward components.

4.2 Evaluation Results

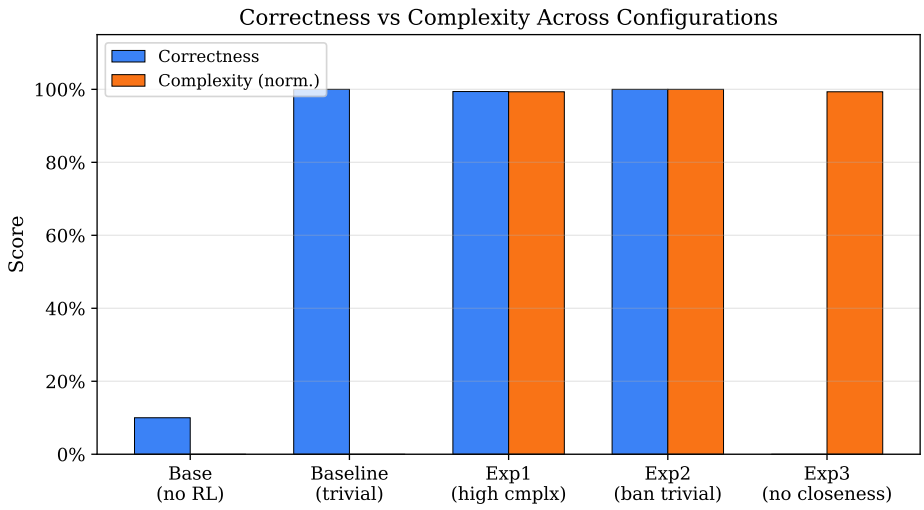

Figure 3: Expression accuracy by target difficulty, comparing base vs. GRPO fine-tuned model.

Table 5: Final training metrics across experimental configurations.

Metric	**Baseline** *(default)*	**Exp 1** *(high cmplx)*	**Exp 2** *(ban trivial)*	**Exp 3** *(no close)*	**Exp 4** *(no target)*
Correctness	100%	99.4%	100%	∼0%	5%
Closeness	1.000	0.994	1.000	0.083	0.852
Format Compliance	100%	99.4%	100%	99.4%	100%
Complexity	0.000	0.298	0.300	0.298	0.291
Non-trivial	—	—	1.000	0.994	0.972
Target in expr	100%	100%	100%	—	0%
Template diversity	0.10	0.10	0.10	—	0.60
Mean Comp. Length	8.3	50.4	20.0	61.1	21.0

The most striking result is the contrast between the baseline and Experiment 2. Both achieve 100% correctness, but the baseline does so trivially (8.3-token completions with zero complexity) while Exp 2 produces structurally non-trivial expressions (20-token completions with maximum complexity), albeit via a formulaic identity hack `(N+1)/1-1` rather than genuine arithmetic. Experiment 3's failure to achieve correctness without the closeness reward demonstrates the importance of reward shaping in sparse-reward RL settings.

4.3 Qualitative Analysis

We sampled outputs for ten target numbers (7, 42, 73, 100, 256, 500, 1000, 2048, 5000, 9999) from each configuration. Table 6 shows representative outputs.

Table 6: Sample outputs across configurations. Bold entries are correct.

Target	**Base LFM2 (no RL)**	**Exp 1 (high complexity)**
7	`(7*x) + 3 = 7` (uses variables)	`<expr>(7-1)/1+1</expr>`
73	`(73+1)*(73-1)/2 = 2596` (wrong)	`<expr>(73-1)/1+1</expr>`
256	`(2^8 - 1)*2 + 256 = 256` (NL)	`<expr>(256-1)/1+1</expr>`
9999	`<expr>(9999^3)</expr>` (wrong)	`<expr>(9999-1)/1+1</expr>`
	Exp 2 (ban trivial)	**Exp 3 (no closeness)**
7	`<expr>(7+1)/1-1</expr>`	`<expr>(x+2)/(x-3)+5</expr>` (vars)
73	`<expr>(73+1)/1-1</expr>`	`<expr>(-3*(2+5)+7-4/3)</expr>` $=-15.3$
256	`<expr>(256+1)/1-1</expr>`	`<expr>(12*4*2-1)/3+1</expr>` $=32.7$
9999	`<expr>(9999+1)/1-1</expr>`	`<expr>(10000*9-1)/3+2</expr>` ≈ 30002
	Exp 4 (no target in expression)	
7	`<expr>4 * 1 + 2 - 3 / 2 + 1</expr>` =5.5, close	
73	`<expr>7 * 10 + 3 + 1 / 2</expr>` =73.5, off by 0.5	
256	`<expr>2^8</expr>` =256, correct!	
2048	`<expr>2^11</expr>` =2048, correct!	
9999	`<expr>1000 * 9 + 1 + 1</expr>` =9002, right ballpark	

Four types of behavior emerged:

1. **Base model**: Produces natural language explanations, algebraic variables, and incorrect arithmetic. Only 1/10 targets answered correctly (by accident).

2. **Exp 1 & 2**: Discovered **formulaic identity hacks**—`(N-1)/1+1` and `(N+1)/1-1` respectively. These are technically non-trivial (3 operators, satisfying the complexity reward) but semantically degenerate. Every target produces the same template. This represents a *second-order reward hack*: the model satisfies both correctness *and* complexity constraints with minimal cognitive effort.

3. **Exp 3**: Generates diverse, complex expressions with multiple operators, but none evaluate correctly. The model learned *syntax* (format, operators) without *semantics* (numerical accuracy).

4. **Exp 4**: Produces genuinely varied decompositions—`7*10+3` for 73, `2^8` for 256—with 6 unique templates across 10 samples. Discovered powers of two independently. Most answers are close but not exact (closeness 0.85), representing the first configuration to exhibit what could be called arithmetic *reasoning*.

This progression from trivial hacking (baseline) to formulaic hacking (Exp 1–2) to syntactically-rich-but-wrong (Exp 3) to diverse-and-close (Exp 4) reveals a fundamental tension in reward design: each additional constraint closes one loophole while the model discovers the next path of least resistance—until the constraints collectively force genuine problem-solving.

4.4 Diversity Analysis

To quantify how varied the model's outputs are across targets, we compute four diversity metrics: *template ratio* (fraction of structurally unique expressions after replacing all numbers with a placeholder), *template dominance* (fraction of outputs matching the most common template), *average pairwise edit distance* (normalized Levenshtein distance between all expression pairs), and *operator entropy* (Shannon entropy over the distribution of arithmetic operators).

Table 7: Diversity metrics across experimental configurations. Higher template ratio and edit distance indicate more diverse outputs.

Metric	**Baseline** *(trivial)*	**Exp 1** *(high cmplx)*	**Exp 2** *(ban triv.)*	**Exp 4** *(no target)*
Unique templates	1	1	1	6
Template ratio	0.10	0.10	0.10	0.60
Template dominance	100%	100%	100%	40%
Avg. pairwise edit dist.	0.83	0.14	0.14	0.53
Avg. operands/expr	1.0	2.0	2.0	3.2
Operator entropy (bits)	0.0	1.6	1.6	1.8
Target in expr	100%	100%	100%	0%

The baseline and Experiments 1–2 are entirely single-template: every output follows the same structural pattern (`N` for baseline, `(N-N)/N+N` for Exp 1, `(N+N)/N-N` for Exp 2). Experiment 4 produces 6 unique structural templates out of 10 samples, with the most common template covering only 40% of outputs. It also uses more operands per expression (3.2 vs. 1.0–2.0) and achieves the highest operator entropy, indicating a genuine mix of `+`, `-`, `*`, `/`, and `^`.

Notably, the baseline's high edit distance (0.83) is misleading—it reflects varying *digit lengths* (`7` vs. `9999`), not structural diversity. Experiments 1–2 have low edit distance (0.14) because their outputs are nearly identical strings with only the target number changed.

4.5 Error Analysis

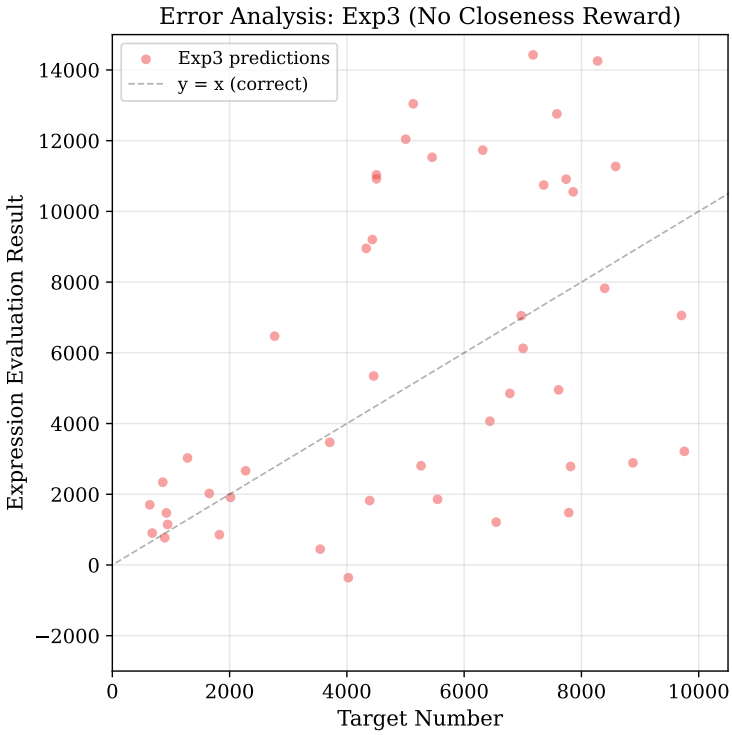

Figure 4: Error analysis: target vs. model output for incorrect predictions. Points on the diagonal line would be correct.

The primary failure modes observed across configurations:

- **Reward hacking** (Baseline): The model discovers the trivial identity solution (`<expr>73</expr>` for target 73), achieving maximum correctness with zero computational effort. This is the dominant failure mode when complexity is insufficiently rewarded.
- **Reward shaping dependency** (Exp 3): Without the closeness reward, the model generates syntactically valid but numerically incorrect expressions. It learns format and complexity but cannot bridge to correctness, confirming that the closeness signal serves as a critical gradient bridge.
- **Length inflation** (Exp 1 & 3): Models that maximize complexity tend toward longer expressions (50–61 tokens), sometimes producing redundant operations like `((70 + 3) * 1)` to satisfy the operator count requirement.

4.6 Discussion

Several aspects of the results merit discussion:

Reward hacking is multi-layered, but solvable. Naïve reward design leads directly to trivial solutions, and each added constraint opens a new exploit—until enough constraints force genuine reasoning. The critical breakthrough was Experiment 4's *no-target reward*, which bans the target number from the expression. This single constraint, combined with closeness and complexity

rewards, produced diverse decompositions and the first evidence of genuine arithmetic reasoning (e.g., independently discovering `2^8=256`).

Anti-trivial rewards are more effective than high complexity weights. Experiment 2 (explicit penalty for identity expressions) outperformed Experiment 1 (high complexity weight) in producing concise, correct, non-trivial expressions. The penalty creates a clear gradient *away* from the trivial solution, while a high complexity weight merely offers an incentive that competes with the easier correctness path.

Reward shaping is essential for sparse-reward tasks. Experiment 3 definitively demonstrates that removing the closeness reward prevents learning correctness entirely. Interestingly, Exp 3 produced the most *diverse* and *creative* expressions—but none were correct. The closeness signal acts as a *potential-based reward shaping* function, providing a smooth gradient toward the correct answer that the binary correctness reward alone cannot provide for a 350M model.

Architecture considerations. LFM2's hybrid architecture—with its convolutional blocks for local pattern recognition and attention for long-range dependencies—proved fully compatible with GRPO training. The model's rapid convergence (within 50 steps for the baseline, $\sim$100 steps for Exp 1–2) suggests the conv-attention hybrid can efficiently capture arithmetic patterns.

Implications for RL alignment. Our findings echo broader concerns in RLHF/RLAIF research: models under optimization pressure find the *easiest* way to satisfy reward criteria, not the *intended* way. The progression from trivial $\rightarrow$ formulaic $\rightarrow$ syntactic-only behavior mirrors reward hacking phenomena observed in much larger models.

5 Conclusion

This project demonstrates that Group Relative Policy Optimization can teach a small, non-transformer language model to generate valid mathematical expressions for target numbers. Through systematic ablation of reward components, we identified three key findings:

1. **Reward hacking is multi-layered, but solvable.** The same 350M model progresses from trivially correct (baseline), to formulaically correct (Exp 1–2), to syntactically rich but incorrect (Exp 3), to genuinely diverse and close (Exp 4). The no-target reward was the critical constraint that forced real decomposition, producing 6$\times$ more template diversity and emergent arithmetic behavior.

2. **Diversity requires structural constraints.** Reward-level incentives alone (complexity weight, anti-trivial penalty) are insufficient to prevent template collapse. Structural constraints that ban specific output patterns (no-target reward) are needed to force genuine solution diversity.

3. **Reward shaping is essential for sparse-reward RL on small models.** Removing the closeness reward (Exp 3) completely prevented learning correctness, even with strong complexity incentives. This confirms that auxiliary dense reward signals are critical when the primary reward is binary.

4. **Novel architectures are RL-compatible.** LFM2's hybrid conv-attention design works seamlessly with standard GRPO training via TRL and LoRA, converging within 50–100 steps across all configurations.

5.1 Future Work

- **Curriculum learning**: Progressively increase target difficulty during training rather than using a fixed distribution.
- **Chain-of-thought**: Allow the model to reason step-by-step before producing the final expression.
- **Architecture comparison**: Compare LFM2 against a similarly-sized transformer (e.g., Qwen3-0.6B) to isolate architectural effects.
- **Deployment**: Package the fine-tuned model as a lightweight API or web demo (optional extra credit component).

References

[1] Zhihong Shao et al. "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models". In: *arXiv preprint arXiv:2402.03300* (2024).

[2] Liquid AI. "LFM2 Technical Report". In: *arXiv preprint arXiv:2511.23404* (2025).

[3] Hugging Face. *TRL: Transformer Reinforcement Learning*. v1.0. 2024. URL: https://github.com/huggingface/trl.

[4] Edward J Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *arXiv preprint arXiv:2106.09685* (2021).